IC253: Data Structures and Algorithm

Assignment 4 Report

Name: Daksh Rathi
Roll No: B25132

1 Introduction

This report presents the implementation of both tasks of Assignment 4. Task 1 implements a smart-city road network system using the class `CityGraph`, while Task 2 implements a logistics dispatch system using the class `DispatchCentre`. The report summarizes the validation and handling policies, the main data structures, the overall implementation approach, the important functions used in the code, the time and space complexity of the operations, and the selected hardcoded test-case outputs used for report presentation.

2 Task 1: Smart City Road Network Optimizer

2.1 Objective

The objective of Task 1 is to process a city road graph in which each road stores both `cost` and `travelTime`. The implementation first validates the graph through `validateGraph()`, then constructs the minimum-cost spanning road network using `buildMST()`, finds the minimum-travel-time route using `shortestPath(S, T)`, compares the MST path against the optimal path using `compareNetworks(S, T)`, and finally identifies MST roads whose removal increases the MST cost using `criticalRoads()`.

2.2 Approach

The complete graph is managed by the class `CityGraph`. Each road is stored in the structure `Edge`, which contains `u`, `v`, `cost`, and `travelTime`. The full road list is stored in `edges`, while the roads accepted into the MST are stored in `mstEdges`. The adjacency structures `adjFull` and `adjMST` are used to represent the full graph and the MST graph using travel-time weights.

The function `validateGraph()` first checks whether the graph satisfies the chosen validation policy. After successful validation, `buildMST()` applies Kruskal's algorithm. The helper `sortEdgeIndicesByCost()` arranges edge indices by increasing `cost`, and the DSU helpers `initDSU()`, `findParent()`, and `unionSets()` are used to decide whether a road should be accepted or rejected. For shortest travel-time queries, `dijkstra()` is used on `adjFull`, and the final route is rebuilt by `reconstructPath()`. The function `compareNetworks(S, T)` evaluates the travel time on both `adjFull` and `adjMST`, while `criticalRoads()` removes one road from `mstEdges` at a time and rebuilds the MST cost to measure the impact.

2.3 Graph Validation Policy

The validation policy enforced by `validateGraph()` is:

- self-loops are not allowed,
- duplicate roads between the same unordered pair are not allowed,
- both `cost` and `travelTime` must be non-negative,
- every node id must lie in the range $[0, N-1]$,
- the graph must be connected, checked through `countReachableFromZero()`.

If any rule fails, the function reports the first specific reason. Invalid values of `S` and `T` are handled inside `shortestPath(S, T)` and `compareNetworks(S, T)`.

2.4 Data Structures Used

Data Structure / Variable	Main Fields / Form	Use in the Implementation
`Edge`	`u`, `v`, `cost`, `travelTime`	Stores one road of the city graph together with the construction cost and travel-time weight.
`edges`	`vector<Edge>`	Stores the original input graph on which validation, MST construction, and full-graph shortest path are performed.
`mstEdges`	`vector<Edge>`	Stores only those roads that are accepted into the MST during `buildMST()`.
`adjFull`, `adjMST`	`vector<vector<pair<int,int>>`	Store the full graph and MST graph in adjacency-list form weighted by `travelTime`.
`parent`, `setSize`	`vector<int>`	DSU arrays used inside Kruskal's algorithm to track connected components and perform union by size.
`dist`, `prev`, `visited`	helper vectors in `dijkstra()`	Used to store shortest distance values, predecessor links, and finalised nodes during shortest-path computation.
`bfsQueue`	`vector<int>`	Used as a simple queue inside `countReachableFromZero()` for the connectivity check.

2.5 Functions and Implementation

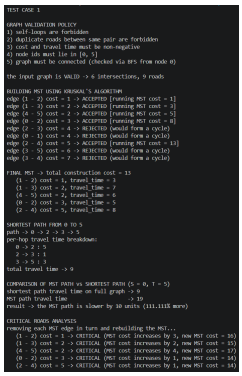
Function Name	Role in the Code
`validateGraph()`	Checks the complete graph-validation policy and prints the first exact reason if the graph is invalid.
`buildMST()`	Sorts roads by `cost`, applies Kruskal's algorithm, prints accept/reject status, updates `mstCost`, and fills `mstEdges`.
`shortestPath(S, T)`	Runs `dijkstra()` on `adjFull`, rebuilds the route through `reconstructPath()`, and prints the per-hop `travelTime` breakdown.
`compareNetworks(S, T)`	Measures the travel time on `adjFull` and `adjMST` through `getPathTravelTime()` and reports how much slower the MST path is.
`criticalRoads()`	Removes one road from `mstEdges` at a time, rebuilds the MST cost, and reports whether that road is critical.
`initDSU()`, `findParent()`, `unionSets()`	Implement the DSU required by Kruskal's algorithm using path compression and union by size.
`sortEdgeIndicesByCost()`	Performs selection sort on the edge-index array so that the original order of `edges` remains unchanged.
`dijkstra()`	Computes shortest travel times from the source by using the simple $O(N^2)$ version of Dijkstra's algorithm.
`reconstructPath()`	Uses the `prev` array to rebuild the final `S` to `T` route after Dijkstra's run.
`getPathTravelTime()`	Returns the shortest travel time between two nodes on a given adjacency structure.
`buildFullAdjacency()`	Builds `adjFull` from `edges`; called once inside the constructor so `shortestPath()` can run without `buildMST()`.
`buildMSTAdjacency()`	Builds `adjMST` from `mstEdges`; called at the end of `buildMST()`.
`countReachableFromZero()`	Performs the BFS-based connectivity check required by `validateGraph()`.

2.6 Time and Space Complexity Analysis

Let N be the number of intersections and M be the number of roads.

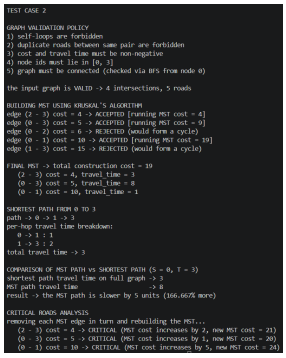
Operation	Time Complexity	Space Complexity	Explanation
`initDSU()`, `findParent()`, `unionSets()`	near $O(1)$ amortized	$O(N)$	DSU uses path compression and union by size.
`sortEdgeIndicesByCost()`	$O(M^2)$	$O(M)$	Selection sort is performed on the edge-index array.
`buildFullAdjacency()`	$O(M)$	$O(N+M)$	Builds `adjFull` from `edges` once in the constructor.
`buildMSTAdjacency()`	$O(N)$	$O(N)$	Builds `adjMST` from `mstEdges` at the end of `buildMST()`.
`countReachableFromZero()`	$O(N+M)$	$O(N+M)$	One BFS over a temporary adjacency list.
`validateGraph()`	$O(M^2)$	$O(N+M)$	Nested duplicate-road checking dominates the validation pass.
`dijkstra()`	$O(N^2+M)$	$O(N)$	The next node is chosen by scanning `dist` linearly.
`buildMST()`	$O(M^2 + M\,\alpha(N))$	$O(N+M)$	Kruskal's algorithm is used, but selection sort dominates.
`shortestPath(S, T)`	$O(N^2+M)$	$O(N)$	One Dijkstra pass plus path reconstruction.
`compareNetworks(S, T)`	$O(N^2+M)$	$O(N)$	Shortest-time queries are performed on `adjFull` and `adjMST`.
`criticalRoads()`	$O(M^2 + NM\,\alpha(N))$	$O(N+M)$	The sorted index list is reused, then the MST is rebuilt once per edge of `mstEdges`.

2.7 Test Case Demonstrations



```
TEST CASE 1

GRAPH VALIDATION POLICY
1) self-loops are forbidden
2) duplicate roads between same pair are forbidden
3) cost and travel time must be non-negative
4) node ids must lie in [0, 5]
5) graph must be connected (checked via BFS from node 0)

the input graph is VALID -> 6 intersections, 9 roads

BUILDING MST USING KRUSKAL'S ALGORITHM
edge (1 - 2) cost = 1 -> ACCEPTED [running MST cost = 1]
edge (1 - 3) cost = 2 -> ACCEPTED [running MST cost = 3]
edge (4 - 5) cost = 2 -> ACCEPTED [running MST cost = 5]
edge (0 - 2) cost = 3 -> ACCEPTED [running MST cost = 8]
edge (2 - 3) cost = 4 -> REJECTED (would form a cycle)
edge (0 - 1) cost = 4 -> REJECTED (would form a cycle)
edge (2 - 4) cost = 5 -> ACCEPTED [running MST cost = 13]
edge (3 - 5) cost = 6 -> REJECTED (would form a cycle)
edge (3 - 4) cost = 7 -> REJECTED (would form a cycle)

FINAL MST -> total construction cost = 15
   (1 - 2) cost = 1, travel_time = 1
   (1 - 3) cost = 2, travel_time = 7
   (4 - 5) cost = 2, travel_time = 6
   (0 - 2) cost = 3, travel_time = 5
   (2 - 4) cost = 5, travel_time = 8

SHORTEST PATH FROM 0 TO 5
path -> 0 -> 2 -> 3 -> 5
per-hop travel time breakdown:
   0 -> 2 : 5
   2 -> 3 : 1
   3 -> 5 : 3
total travel time -> 9

COMPARISON OF MST PATH vs SHORTEST PATH (S = 0, T = 5)
shortest path travel time on full graph -> 9
MST path travel time                    -> 19
result -> the MST path is slower by 10 units (111.111% more)

CRITICAL ROADS ANALYSIS
removing each MST edge in turn and rebuilding the MST...
   (1 - 2) cost = 1 -> CRITICAL (MST cost increases by 5, new MST cost = 20)
   (1 - 3) cost = 2 -> CRITICAL (MST cost increases by 2, new MST cost = 15)
   (4 - 5) cost = 2 -> CRITICAL (MST cost increases by 4, new MST cost = 17)
   (0 - 2) cost = 3 -> CRITICAL (MST cost increases by 1, new MST cost = 14)
   (2 - 4) cost = 5 -> CRITICAL (MST cost increases by 1, new MST cost = 14)
```

(a) Task 1 - Test Case 1 Output



```
TEST CASE 2

GRAPH VALIDATION POLICY
1) self-loops are forbidden
2) duplicate roads between same pair are forbidden
3) cost and travel time must be non-negative
4) node ids must lie in [0, 3]
5) graph must be connected (checked via BFS from node 0)

the input graph is VALID -> 4 intersections, 5 roads

BUILDING MST USING KRUSKAL'S ALGORITHM
edge (2 - 3) cost = 4 -> ACCEPTED [running MST cost = 4]
edge (0 - 3) cost = 5 -> ACCEPTED [running MST cost = 9]
edge (0 - 2) cost = 6 -> REJECTED (would form a cycle)
edge (0 - 1) cost = 10 -> ACCEPTED [running MST cost = 19]
edge (1 - 3) cost = 15 -> REJECTED (would form a cycle)

FINAL MST -> total construction cost = 19
   (2 - 3) cost = 4, travel_time = 3
   (0 - 3) cost = 5, travel_time = 8
   (0 - 1) cost = 10, travel_time = 1

SHORTEST PATH FROM 0 TO 3
path -> 0 -> 1 -> 3
per-hop travel time breakdown:
   0 -> 1 : 1
   1 -> 3 : 2
total travel time -> 3

COMPARISON OF MST PATH vs SHORTEST PATH (S = 0, T = 3)
shortest path travel time on full graph -> 3
MST path travel time                    -> 8
result -> the MST path is slower by 5 units (166.667% more)

CRITICAL ROADS ANALYSIS
removing each MST edge in turn and rebuilding the MST...
   (2 - 3) cost = 4 -> CRITICAL (MST cost increases by 2, new MST cost = 21)
   (0 - 3) cost = 5 -> CRITICAL (MST cost increases by 1, new MST cost = 20)
   (0 - 1) cost = 10 -> CRITICAL (MST cost increases by 5, new MST cost = 24)
```

(b) Task 1 - Test Case 2 Output

Figure 1: Task 1 selected test cases

3 Task 2: Logistics Dispatch Centre

3.1 Objective

The objective of Task 2 is to manage a live dispatch queue in which the next processed request must always have the highest `priority`, while still supporting update operations and shift-end sorting. The implementation validates the request list using `validateInput(requests)`, builds the live structure using `buildStructure(requests)`, removes the highest-priority request using `dispatchNext()`, changes an existing request by `updatePriority(request_id, new_priority)`, sorts records using `sortRecords(records, key)`, and returns the highest-priority requests through `topKRequests(k)` without changing the live heap.

3.2 Approach

The complete implementation is managed by the class `DispatchCentre`. Each request is stored in the structure `Request`, which contains `requestID`, `priority`, and `timestamp`. The live queue is stored in the vector `heap`, treated as an array-based max-heap. The helper `buildStructure(requests)` copies the input requests into `heap` and restores the heap property through bottom-up `siftDown()` calls.

Once the structure is built, `dispatchNext()` removes the root request and restores the heap property by moving the replacement downward. The function `updatePriority(request_id, new_priority)` first finds the request using `findIndexByID(request_id)` and then calls either `siftUp()` or `siftDown()` depending on whether the priority increased or decreased. For shift-end record handling, `sortRecords(records, key)` calls `quicksortHelper(arr, lo, hi, key, partitionCount)` to sort the given vector in-place. The function `topKRequests(k)` does not modify `heap`; instead, it copies the heap into `tempHeap` and extracts the maximum k times from the copy.

3.3 Data Handling Policy

The policy enforced by `validateInput(requests)` and `updatePriority(request_id, new_priority)` is:

- every `requestID` must be unique inside one input batch,
- every `priority` must lie in the range $[0, 100]$,
- an `updatePriority()` call is invalid if the id is unknown,
- an `updatePriority()` call is invalid if the new priority is outside $[0, 100]$.

The code prints the exact invalid reason and leaves the live structure unchanged when an invalid update is attempted.

3.4 Data Structures Used

Data Structure / Variable	Main Fields / Form	Use in the Implementation
`Request`	`requestID`, `priority`, `timestamp`	Stores one aid request together with the two keys used for live dispatch and shift-end sorting.
`heap`	`vector<Request>`	Stores the live dispatch structure as an array-based max-heap ordered by `priority`.
`tempHeap`	local `vector<Request>`	Stores a copy of `heap` inside `topKRequests(k)` so that the live structure is not modified.
`parent`, `leftChild`, `rightChild`	integer indices	Used inside heap operations to move upward or downward while restoring the heap property.
`lo`, `hi`, `pivIdx`, `partitionCount`	quicksort helper variables	Used to select the pivot, define the current subarray, and print recursive partition information.

3.5 Functions and Implementation

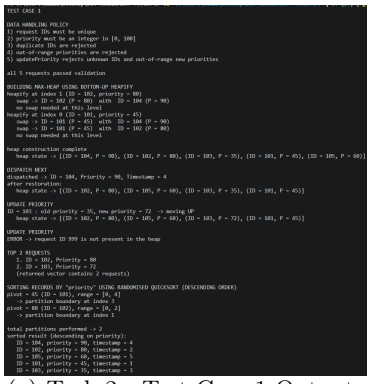
Function Name	Role in the Code
`validateInput(requests)`	Checks duplicate `requestID` values and out-of-range `priority` values before the live structure is built.
`buildStructure(requests)`	Copies the given requests into `heap` and restores the max-heap property through bottom-up heapify.
`dispatchNext()`	Removes the maximum-priority request from the root, restores the heap by `siftDown()`, and prints the updated state.
`updatePriority(request_id, new_priority)`	Finds the target request by `findIndexByID()`, updates its `priority`, and restores the heap by either `siftUp()` or `siftDown()`.
`sortRecords(records, key)`	Sorts the given vector in-place on the selected key by calling `quicksortHelper()`.
`topKRequests(k)`	Returns the k highest-priority requests in decreasing order by repeatedly extracting from `tempHeap`.
`swapNodes()`	Swaps two heap elements during heap maintenance operations.
`siftUp()`, `siftDown()`	Restore the max-heap property after insertion-style or deletion-style priority changes.
`findIndexByID()`	Performs a linear scan on `heap` to locate the request whose `requestID` matches the given id.
`quicksortHelper(arr, lo, hi, key, partitionCount)`	Performs recursive randomized quicksort and prints the pivot and partition boundaries for tracing.

3.6 Time and Space Complexity Analysis

Let n be the number of requests currently stored in the structure.

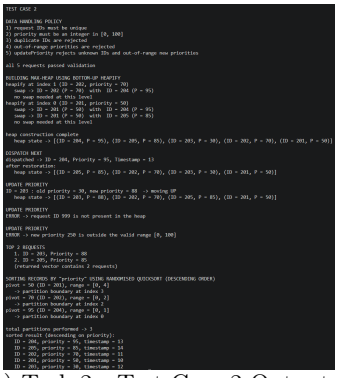
Operation	Time Complexity	Space Complexity	Explanation
`swapNodes()`	$O(1)$	$O(1)$	Swaps two requests using one temporary variable.
`siftUp()`, `siftDown()`	$O(\log n)$	$O(1)$	Only one heap path is traversed.
`findIndexByID()`	$O(n)$	$O(1)$	Request ids are searched through a linear scan on `heap`.
`validateInput()`	$O(n^2)$	$O(1)$	Duplicate ids are checked by comparing each request with all earlier requests.
`buildStructure()`	$O(n)$	$O(n)$	Bottom-up heapify is used after copying the requests into `heap`.
`dispatchNext()`	$O(\log n)$	$O(1)$	Root removal followed by one `siftDown()` call.
`updatePriority()`	$O(n)$	$O(1)$	The linear scan in `findIndexByID()` dominates the update.
`sortRecords()` / `quicksortHelper()`	average $O(n \log n)$, worst $O(n^2)$	average $O(\log n)$	Randomized in-place quicksort uses only recursion stack space on average.
`topKRequests(k)`	$O(n + k \log n)$	$O(n)$	A copy of `heap` is made in `tempHeap`, then extract-max is applied k times.

3.7 Test Case Demonstrations



```
TEST CASE 1

DATA HANDLING POLICY
1) request IDs must be unique
2) priority must be an integer in [0, 100]
3) duplicate IDs are rejected
4) out-of-range priorities are rejected
5) updatePriority rejects unknown IDs and out-of-range new priorities

all 5 requests passed validation

BUILDING MAX-HEAP USING BOTTOM-UP HEAPIFY
heapify at index 1 (ID = 102, priority = 80)
   swap -> ID = 102 (P = 80)  with  ID = 104 (P = 90)
   no swap needed at this level
heapify at index 0 (ID = 101, priority = 45)
   swap -> ID = 101 (P = 45)  with  ID = 104 (P = 90)
   swap -> ID = 101 (P = 45)  with  ID = 102 (P = 80)
   no swap needed at this level

heap construction complete
   heap state -> [(ID = 104, P = 90), (ID = 102, P = 80), (ID = 103, P = 35), (ID = 101, P = 45), (ID = 105, P = 60)]

DISPATCH NEXT
dispatched -> ID = 104, Priority = 90, Timestamp = 4
after restoration:
   heap state -> [(ID = 102, P = 80), (ID = 105, P = 60), (ID = 103, P = 35), (ID = 101, P = 45)]

UPDATE PRIORITY
ID = 103 : old priority = 35, new priority = 72  -> moving UP
   heap state -> [(ID = 102, P = 80), (ID = 105, P = 60), (ID = 103, P = 72), (ID = 101, P = 45)]

UPDATE PRIORITY
ERROR -> request ID 999 is not present in the heap

TOP 2 REQUESTS
   1. ID = 102, Priority = 80
   2. ID = 103, Priority = 72
   (returned vector contains 2 requests)

SORTING RECORDS BY "priority" USING RANDOMISED QUICKSORT (DESCENDING ORDER)
pivot = 45 (ID = 101), range = [0, 4]
   -> partition boundary at index 3
pivot = 80 (ID = 102), range = [0, 2]
   -> partition boundary at index 1

total partitions performed -> 2
sorted result (descending on priority):
   ID = 104, priority = 90, timestamp = 4
   ID = 102, priority = 80, timestamp = 2
   ID = 105, priority = 60, timestamp = 5
   ID = 101, priority = 45, timestamp = 1
   ID = 103, priority = 35, timestamp = 3
```

(a) Task 2 - Test Case 1 Output



```
TEST CASE 2

DATA HANDLING POLICY
1) request IDs must be unique
2) priority must be an integer in [0, 100]
3) duplicate IDs are rejected
4) out-of-range priorities are rejected
5) updatePriority rejects unknown IDs and out-of-range new priorities

all 5 requests passed validation

BUILDING MAX-HEAP USING BOTTOM-UP HEAPIFY
heapify at index 1 (ID = 202, priority = 70)
   swap -> ID = 202 (P = 70)  with  ID = 204 (P = 95)
   no swap needed at this level
heapify at index 0 (ID = 201, priority = 50)
   swap -> ID = 201 (P = 50)  with  ID = 204 (P = 95)
   swap -> ID = 201 (P = 50)  with  ID = 205 (P = 85)
   no swap needed at this level

heap construction complete
   heap state -> [(ID = 204, P = 95), (ID = 205, P = 85), (ID = 203, P = 30), (ID = 202, P = 70), (ID = 201, P = 50)]

DISPATCH NEXT
dispatched -> ID = 204, Priority = 95, Timestamp = 11
after restoration:
   heap state -> [(ID = 205, P = 85), (ID = 202, P = 70), (ID = 203, P = 30), (ID = 201, P = 50)]

UPDATE PRIORITY
ID = 203 : old priority = 30, new priority = 88  -> moving UP
   heap state -> [(ID = 203, P = 88), (ID = 202, P = 70), (ID = 205, P = 85), (ID = 201, P = 50)]

UPDATE PRIORITY
ERROR -> request ID 999 is not present in the heap

UPDATE PRIORITY
ERROR -> new priority 250 is outside the valid range [0, 100]

TOP 2 REQUESTS
   1. ID = 203, Priority = 88
   2. ID = 205, Priority = 85
   (returned vector contains 2 requests)

SORTING RECORDS BY "priority" USING RANDOMISED QUICKSORT (DESCENDING ORDER)
pivot = 50 (ID = 201), range = [0, 4]
   -> partition boundary at index 1
pivot = 70 (ID = 202), range = [0, 2]
   -> partition boundary at index 1
pivot = 95 (ID = 204), range = [0, 1]
   -> partition boundary at index 0

total partitions performed -> 3
sorted result (descending on priority):
   ID = 204, priority = 95, timestamp = 13
   ID = 205, priority = 85, timestamp = 14
   ID = 202, priority = 70, timestamp = 11
   ID = 203, priority = 50, timestamp = 10
   ID = 201, priority = 30, timestamp = 12
```

(b) Task 2 - Test Case 2 Output

Figure 2: Task 2 selected test cases

4 GitHub Repository

The complete source code and associated test cases for both Task 1 and Task 2 are available in the following repository:

- `https://github.com/dakshrathi-india/DSA-ASSIGNMENT-4`